

第五章附录：实战代码详细讲解

这份附录是第 5 章实战 notebook（05.ipynb）的逐行导读，面向不太熟悉 Python / PyTorch 的读者：按代码的功能块讲清楚每个 cell 做了什么，入参、出参、中间变量分别是什么，形状怎么一步步变。正文的「为什么这样设计」在第 5 章：从 RNN 到 attention 里，这里专注「代码每一行在干什么」，两份对照着读最顺。

想直接跑示例？点这里  [Open in Colab](#)。

硬件门槛：概念章，CPU 即可 ✓。实战训两个很小的 GRU seq2seq（隐藏维 96、序列长 ≤ 12），在 Colab 免费 CPU 运行时上约 2-4 分钟跑完，不需要 GPU。

整本 notebook 的主线（Cell 0-1 是环境自检 + 装依赖，无须细讲，本附录略过）：

Cell 2：手算 RNN 递推——理解「隐藏态滚动历史」

这个 cell 不训练任何东西，纯粹用一个示例尺寸手动走三步 RNN，让你看清隐藏状态 h 是怎么把历史累积进去的。

```
import torch

torch.manual_seed(0)
d_in, d_h = 3, 2                                     # 输入 3 维、隐藏 2 维
W_xh = torch.randn(d_h, d_in)                         # [d_h, d_in] 输入到隐藏
W_hh = torch.randn(d_h, d_h)                         # [d_h, d_h] 隐藏到隐藏（这就是「循环」的那条边）
b = torch.zeros(d_h)

def rnn_step(x_t, h_prev):
    """单步 RNN：把上一步隐藏 h_prev 和当前输入 x_t 揉进新的隐藏 h_t。"""
    return torch.tanh(W_xh @ x_t + W_hh @ h_prev + b)

xs = [torch.randn(d_in) for _ in range(3)]           # 一条长度 3 的序列
h = torch.zeros(d_h)                                 # 初始隐藏状态置 0
for t, x_t in enumerate(xs):
    h = rnn_step(x_t, h)                             # 同一组权重 W_xh/W_hh 反复用于每一步
    print(f"第 {t} 步后的隐藏状态 h = {h.numpy().round(3)}")
```

RNN 的核心就一条递推： $h_t = \tanh(W_{hh} h_{t-1} + W_{xh} x_t + b)$ 。其中 x_t 是第 t 步的输入， h_{t-1} 是上一步的隐藏态， $\tanh$ 是把数值压到 $(-1, 1)$ 的激活函数。

权重（在循环外只建一次）：

变量	形状	含义
<code>W_xh</code>	<code>[d_h, d_in] = [2, 3]</code>	输入 → 隐藏的变换
<code>W_hh</code>	<code>[d_h, d_h] = [2, 2]</code>	隐藏 → 隐藏，这条「自己连回自己」的边就是「循环」
<code>b</code>	<code>[d_h] = [2]</code>	偏置

`rnn_step` 函数：入参 `x_t`（当前输入，`[3]`）和 `h_prev`（上一步隐藏，`[2]`），出参是新隐藏 `h_t`（`[2]`）。`@` 是矩阵乘法，`w_xh @ x_t` 得 `[2]`，`w_hh @ h_prev` 得 `[2]`，相加过 `tanh`。

循环：`xs` 是 3 个随机输入向量组成的序列（每个 `[3]`），`h` 初始化为全 0。每一步用同一组权重 `w_xh` / `w_hh` 把新输入和旧隐藏揉成新隐藏。`enumerate(xs)` 让循环同时拿到下标 `t` 和元素 `x_t`。

一句话总结：Cell 2 用一个 2 维隐藏、3 维输入的示例，手动走三步 RNN。关键直觉是——每一步都复用同一套权重，`h` 把「到目前为止读过的所有 token」压成一个定长向量。这正是后面 GRU 编码器的雏形。

Cell 3: 造「数字串 → 逆序」数据 + 词表

这段代码干一件事：生成「数字串 → 逆序」这个示例任务的训练数据，并定义词表。它本身不训练任何东西，只负责批量生产训练数据。

一、词表定义（4 行常量）

```
PAD, SOS, EOS = 0, 1, 2      # 三个特殊 token
DIGIT0 = 3                  # 数字 0..9 映射到 token 3..12
VOCAB = 3 + 10              # 词表大小 = 13
```

模型不认识「数字」或「字符」，只认识整数 ID（token）。所以先约定一张映射表：

含义	token ID	说明
PAD	0	填充符。一个 batch 里序列长短不一，短的右边补 PAD 凑齐成方阵
SOS	1	Start Of Sequence，解码器起步的「发令枪」
EOS	2	End Of Sequence，告诉模型「输出到此结束」
数字 0-9	3-12	数字 <code>d</code> 的 token = <code>d + DIGIT0</code> = <code>d + 3</code>

所以 `VOCAB = 13`：模型最终要在这 13 个 ID 上做分类。这几个都是普通整数常量，没有形状。

Python 小知识：`PAD, SOS, EOS = 0, 1, 2` 是「多重赋值」，一行同时给三个变量赋值，等价于分三行写。

二、make_batch 函数——批量生产训练数据

```
def make_batch(batch_size, min_len=4, max_len=12, seed=None):
    g = torch.Generator().manual_seed(seed) if seed is not None else None
    lens = torch.randint(min_len, max_len + 1, (batch_size,), generator=g)
    Smax = int(lens.max())
    src = torch.full((batch_size, Smax), PAD, dtype=torch.long)
    tgt_in = torch.full((batch_size, Smax + 1), PAD, dtype=torch.long)
```

```

tgt_out = torch.full((batch_size, Smax + 1), PAD, dtype=torch.long)
for i, L in enumerate(lens.tolist()):
    digits = torch.randint(0, 10, (L,), generator=g) + DIGIT0 # L 个数字 token
    rev = torch.flip(digits, dims=[0]) # 逆序
    src[i, :L] = digits
    tgt_in[i, 0] = SOS
    tgt_in[i, 1:L + 1] = rev
    tgt_out[i, :L] = rev
    tgt_out[i, L] = EOS
return src, lens, tgt_in, tgt_out

```

入参（函数括号里的东西）：

入参	类型	含义
batch_size	int	这一批生成几条样本（记作 B）
min_len / max_len	int	每条数字串的长度在 [4, 12] 间随机取
seed	int 或 None	随机种子。给定同一个 seed 每次生成完全一样的数据（可复现）；None 每次都不同

带 = 的是默认参数：调用时不传就用默认值。make_batch(3) 等于 make_batch(3, min_len=4, max_len=12, seed=None)。

出参（return 返回的东西）：一次返回 4 个张量。设 B = batch_size, Smax = 这批里最长那条数字串的长度：

出参	形状	含义
src	[B, Smax]	编码器输入：原始数字串的 token，右侧补 PAD
src_len	[B]	每条源串的真实长度（不含 PAD），后面 GRU 要靠它跳过 PAD
tgt_in	[B, Smax+1]	解码器输入：SOS + 逆序数字（teacher forcing 用）
tgt_out	[B, Smax+1]	解码器目标：逆序数字 + EOS（算 loss 用）

注意 tgt_in / tgt_out 的第二维是 Smax+1，比 src 多 1——因为要多塞一个 SOS（或 EOS）。

函数体逐段拆：

第 1 段：决定每条样本多长。

```

g = torch.Generator().manual_seed(seed) if seed is not None else None
lens = torch.randint(min_len, max_len + 1, (batch_size,), generator=g)
Smax = int(lens.max())

```

`g` 是随机数生成器（给了 `seed` 就造一个可复现的 `g`，否则 `None`，这行用的是三元表达式 `A if 条件 else B`）。`lens` 是形状 `[B]` 的整数张量，每个元素是该样本长度，比如 `[6, 4, 5]`。`Smax` 是这批最大长度，决定补齐后矩阵有多宽。

第 2 段：先开好三个「全填 PAD」的空白张量。

```
src      = torch.full((batch_size, Smax),    PAD, dtype=torch.long)
tgt_in   = torch.full((batch_size, Smax + 1), PAD, dtype=torch.long)
tgt_out  = torch.full((batch_size, Smax + 1), PAD, dtype=torch.long)
```

`torch.full(形状, 填充值)` 造指定形状、全填 `PAD(=0)` 的张量。先全铺 `PAD`、再往里填真实内容，没填到的地方自然就是 `PAD`——这是补齐的常用套路。`dtype=torch.long` 表示 64 位整数（token ID 必须是整数）。此刻 `src` 是 `[B, Smax]`、两个 `tgt` 是 `[B, Smax+1]`，全是 0。

第 3 段：逐条样本填真实数据（核心循环）。

```
for i, L in enumerate(lens.tolist()):
    digits = torch.randint(0, 10, (L,), generator=g) + DIGIT0  # L 个数字 token
    rev = torch.flip(digits, dims=[0])                        # 逆序
    src[i, :L] = digits
    tgt_in[i, 0] = SOS
    tgt_in[i, 1:L + 1] = rev
    tgt_out[i, :L] = rev
    tgt_out[i, L] = EOS
```

`enumerate` 让循环同时拿到下标 `i`（第几条）和长度 `L`：

- `digits` (`[L]`)：随机生成 `L` 个 0–9 的数字，`+ DIGIT0` 整体平移成 token（数字 5 → token 8）。
- `rev` (`[L]`)：`torch.flip(..., dims=[0])` 把 `digits` 沿第 0 维翻转，就是逆序，也就是「答案」。
- `src[i, :L] = digits`：把原数字填进第 `i` 行的前 `L` 列；第 `L` 列往后保持 `PAD`。（`[i, :L]` 是切片，意思「第 `i` 行、第 0 到 `L-1` 列」。）
- `tgt_in[i, 0] = SOS`：解码器输入第一个位置放 `SOS`。
- `tgt_in[i, 1:L+1] = rev`：紧跟 `SOS` 之后放逆序数字。所以 `tgt_in` = `[SOS, 逆序...]`。
- `tgt_out[i, :L] = rev`：目标前 `L` 个就是逆序数字。
- `tgt_out[i, L] = EOS`：第 `L` 个位置放 `EOS`。所以 `tgt_out` = `[逆序..., EOS]`。

为什么 `tgt_in` 和 `tgt_out` 要错位一格？这是 teacher forcing 的精髓：解码器在第 `t` 步读 `tgt_in[t]`，要预测 `tgt_out[t]`。两者错开一位，等于训练模型「看着 `SOS` 猜第一个逆序数字、看着第一个逆序数字猜第二个……最后看着最后一个数字猜 `EOS`」。举个具体例子，原串 `3 1 4`：

```
tgt_in :  SOS  4   1   3       ← 喂给解码器
tgt_out:  4   1   3   EOS      ← 期望它输出（每个位置都比输入超前一步）
```

三、最后的演示打印

```
src, src_len, tgt_in, tgt_out = make_batch(3, min_len=4, max_len=6, seed=42)
print("src      ...", src)
print("src_len ...", src_len.tolist())
print("tgt_in   ...", tgt_in)
print("tgt_out  ...", tgt_out)
```

调一次 `make_batch` 造 3 条样本（长度 4–6，固定 `seed` 便于复现），把 4 个返回值打印出来肉眼检查。`.tolist()` 把张量转成普通 Python 列表，打印更清爽。这一步纯粹是「自检」——确认 `src` 是原串、`tgt_in` 以 1(SOS) 开头、`tgt_out` 以 2(EOS) 结尾。

一句话总结：Cell 3 = 定义 13 个 token 的词表 + 写一个 `make_batch` 函数批量生产「数字串 → 逆序」样本。关键设计有两个：右侧补 PAD 把变长序列对齐成方阵，以及 `tgt_in / tgt_out` 错位一格实现 teacher forcing。

Cell 4: 编码器（GRU + 跳过 PAD）

这段代码定义 Encoder（编码器）：用一个 GRU 把源数字串「读」成一串隐藏状态，供后面解码器和 attention 使用。

先交代两个背景概念：

- GRU（Gated Recurrent Unit，门控循环单元）：RNN 的一种改良。它从左到右逐个 token 读序列，每读一个就更新一个隐藏状态 `h`——这个 `h` 是定长向量，可理解为「把读过的内容压缩成的记忆」。Cell 2 手算的那条递推就是最朴素的 RNN，GRU 在它基础上加了「门」来更好地保留长期信息。
- Embedding（嵌入）：token 是整数 ID，无法直接做数学运算。Embedding 是一张可学习的查找表，把每个 token ID 映射成一个稠密向量（比如 32 维）。

一、`__init__`：定义组件

```
class Encoder(nn.Module):
    def __init__(self, vocab, emb_dim, hid_dim):
        super().__init__()
        self.embed = nn.Embedding(vocab, emb_dim, padding_idx=PAD)
        self.gru = nn.GRU(emb_dim, hid_dim, batch_first=True)
```

`class Encoder(nn.Module)` 表示定义一个神经网络模块（PyTorch 里所有模型都继承 `nn.Module`）。

`__init__` 是「构造函数」，在 `enc = Encoder(...)` 那一刻执行一次，作用是把要用到的零件先创建好。

入参（构造时传的超参数）：

入参	含义
<code>vocab</code>	词表大小，这里是 13。决定 Embedding 表有几行
<code>emb_dim</code>	每个 token 嵌入成多少维向量，这里是 32
<code>hid_dim</code>	GRU 隐藏状态的维度，这里是 96

创建的两个零件：

- `self.embed = nn.Embedding(13, 32, padding_idx=PAD)`：一张 `13 × 32` 的查找表。
`padding_idx=PAD` 是关键——它让 `PAD(=0)` 这个 token 的向量恒为全 0 且不参与梯度更新，因为 `PAD` 是凑数的，不该有语义。
- `self.gru = nn.GRU(32, 96, batch_first=True)`：输入 32 维、隐藏 96 维的 GRU。
`batch_first=True` 表示输入张量形状约定为 `[B, S, ...]`（batch 维在最前），不写的话默认是 `[S, B, ...]`，容易绕晕。

`self`。开头的变量是「实例属性」，存在对象上，`forward` 里还能取用。`super().__init__()` 是固定写法，必须调，负责初始化父类 `nn.Module` 的内部机制。

二、forward：真正的前向计算

`forward` 定义「数据进来后怎么算」。当你写 `enc(src, src_len)` 时，PyTorch 会自动调用这个 `forward`。

```
def forward(self, src, src_len):
    emb = self.embed(src) # [B, S, emb]
    packed = nn.utils.rnn.pack_padded_sequence(
        emb, src_len.cpu(), batch_first=True, enforce_sorted=False)
    packed_out, h_n = self.gru(packed)
    outputs, _ = nn.utils.rnn.pad_packed_sequence(
        packed_out, batch_first=True, total_length=src.size(1))
    return outputs, h_n
```

入参：

入参	形状	含义
<code>src</code>	<code>[B, S]</code>	编码器输入（数字 token，右侧补 PAD），来自 <code>make_batch</code>
<code>src_len</code>	<code>[B]</code>	每条序列的真实长度（不含 PAD）

这里 `S = Smax`，即这批里最长序列的长度。

第 1 步：查表，token → 向量。

```
emb = self.embed(src) # [B, S, emb]
```

`emb = self.embed(src)`：把 `[B, S]` 的整数矩阵，每个 token 替换成它的 32 维向量，得到 `emb`，形状 `[B, S, 32]`。

第 2 步：打包，让 GRU 跳过 PAD（本 cell 的精华）。

```
packed = nn.utils.rnn.pack_padded_sequence(  
    emb, src_len.cpu(), batch_first=True, enforce_sorted=False)
```

问题在于：一个 batch 被补 PAD 凑成了方阵 `[B, S, 32]`，但每条真实长度不同。若直接把整个方阵喂给 GRU，它会连补的 PAD 也一并读一遍，于是：① 最后一步隐藏态 `h_n` 变成「又空跑了一串 PAD 之后」的记忆，被污染；② 污染程度还随 batch 里最长序列变化而漂移——同一条短序列，跟不同长度的序列组 batch，结果都不一样，模型学偏，短序列表现直接变差。

`pack_padded_sequence` 把方阵「压缩」成紧凑结构 `packed`，内部记录每条真实长度，让 GRU 到真实长度处就停，绝不空跑 PAD。其中 `src_len.cpu()` 是长度信息（必须放 CPU），`enforce_sorted=False` 允许 batch 内长度不必预先按降序排好。

第 3 步：GRU 真正跑一遍。

```
packed_out, h_n = self.gru(packed)
```

`packed_out, h_n = self.gru(packed)`：返回两个东西——`packed_out`（每个时间步的隐藏态，仍是打包格式，attention 要用全部位置）；`h_n`（每条序列真实最后一个 token 的隐藏态，形状 `[1, B, 96]`，那个 1 是 GRU 层数，这里单层）。无 attention 的 baseline 就拿 `h_n` 当整句的全部 context。

第 4 步：解包，还原成方阵。

```
outputs, _ = nn.utils.rnn.pad_packed_sequence(  
    packed_out, batch_first=True, total_length=src.size(1))
```

`pad_packed_sequence` 是第 2 步的逆操作，把紧凑的 `packed_out` 还原成规整的 `[B, S, 96]` 张量 `outputs`，PAD 位置补 0。`total_length=src.size(1)` 强制还原到原始宽度 S（不写的话若这批最长真实序列比 S 短，输出宽度会缩水，后面跟 mask 对不齐）。返回值第二个是各序列长度，用不上，用 `_` 接收并丢弃（Python 惯例：`_` 表示「我不关心这个值」）。PAD 位置补的 0 不要紧，后面 attention 会再用 mask 屏蔽掉。

出参：

出参	形状	含义
<code>outputs</code>	<code>[B, S, 96]</code>	每个源位置的隐藏状态，全序列。attention 要逐位置回看它
<code>h_n</code>	<code>[1, B, 96]</code>	每条序列真实最后一个 token 的隐藏态。当解码器初始状态 / baseline 的固定 context

三、形状自检

```
enc = Encoder(VOCAB, emb_dim=32, hid_dim=96)
_out, _h = enc(src, src_len)
print("encoder outputs:", _out.shape, " (= [B, S, hid])")
print("encoder h_n:      ", _h.shape, " (= [1, B, hid] ...)")
```

造一个 Encoder 实例，喂入上面留下的 `src` / `src_len`，把两个输出形状打印确认。注意 `enc(src, src_len)` 这种「像调用函数一样调用对象」的写法，背后就是自动触发 `forward`。变量名带下划线前缀 `_out` / `_h` 提示「这只是临时自检用的」。

一句话总结：Cell 4 = 定义编码器：Embedding 把 token 查成向量 → GRU 逐位读成隐藏状态。核心技巧是用 `pack_padded_sequence` / `pad_packed_sequence` 这对函数让 GRU 在真实长度处停下、不空跑 PAD。产出 `outputs`（全位置隐藏态，给 attention）和 `h_n`（最后一步隐藏态，给解码器起步、给 baseline 当固定 context）。

三个 PyTorch / Python 细节补充：

- 1. 为什么 `enc(src, src_len)` 会自动触发 `forward`？这是 Python 的 `__call__` 机制：
对象(...) 会被翻译成 对象.__call__(...)。nn.Module 已替你实现好 `__call__`，它内部会去调你写的 `forward`（还顺带跑前后置钩子 hooks）。所以链路是 `enc(src, src_len)` → `nn.Module.__call__` → `self.forward(src, src_len)`。别直接写 `enc.forward(...)`——那样会绕过钩子，官方规范一律用 `enc(...)`。
- 2. `batch_first=True` 是什么？它决定张量三个维度的排列顺序。GRU 默认约定 `[S, B, 特征]`（时间步在前，RNN 内部按时间步循环），但人类更习惯 `[B, S, 特征]`，`make_batch` 产出的也是这个顺序。设成 `True` 就让 GRU 按 `[B, S, 特征]` 收发，全流程维度统一，省得反复 `transpose`。注意 `pack_padded_sequence` / `pad_packed_sequence` 也都带了 `batch_first=True`，前后必须一致。
- 3. `total_length=src.size(1)` 是什么？`src` 形状 `[B, S]`，`.size(1)` 取第 1 维大小，即 S（这批补齐后的宽度，等于这批的最大真实长度 Smax）。`pad_packed_sequence` 默认只还原到「当前 batch 最长真实长度」，万一它比 S 小，`outputs` 宽度就会缩水，而 Cell 7 造 mask 用的是 `src.size(1)=S`，两者维度对不上会报错。`total_length` 是个保险：强制解包后的输出宽度恒等于输入宽度 S。

Cell 5: Bahdanau 注意力模块

这是本章的核心——它让解码器每一步都能「回看」整个源序列，动态决定该关注哪些位置。

先把概念立起来：

- Attention（注意力）：解码器生成第 `t` 个输出时，不是只靠一个固定记忆向量，而是回头看编码器输出的所有位置的隐藏态，给每个位置打一个「该关注多少」的权重，再按权重加权求和得到一个为这一步量身定制的 context 向量。
- Bahdanau attention（加性注意力）：打分函数用一个小神经网络（一个 `tanh` + 线性层）算出来。「加性」指 Ws 和 Uh_j 是相加再过 `tanh`（对比后来 Transformer 的「乘性 / 点积」注意力）。打分公式：

$$\text{score}(s, h_j) = v^\top \tanh(Ws + Uh_j)$$

其中 s 是解码器上一步的隐藏态， h_j 是编码器第 j 个位置的隐藏态，三个可学习参数是 $W / U / v$ 。

一、__init__：定义三个线性层

```
class BahdanauAttention(nn.Module):
    def __init__(self, hid_dim):
        super().__init__()
        self.W = nn.Linear(hid_dim, hid_dim, bias=False) # 作用在 decoder 隐藏 s 上
        self.U = nn.Linear(hid_dim, hid_dim, bias=False) # 作用在 encoder 每个 h_j 上
        self.v = nn.Linear(hid_dim, 1, bias=False) # 把打分压成一个标量
```

入参： `hid_dim` ——隐藏维度，这里是 96。

`nn.Linear` 是什么： 一个全连接层（线性变换）， `nn.Linear(in, out)` 把 `in` 维向量乘一个可学习矩阵变成 `out` 维。`bias=False` 表示不加偏置项（只做矩阵乘，不加常数）。

三个零件对应公式里的三个参数：

零件	形状变换	对应公式	作用
<code>self.W</code>	$96 \rightarrow 96$	Ws	变换解码器隐藏态 s
<code>self.U</code>	$96 \rightarrow 96$	Uh_j	变换编码器每个位置 h_j
<code>self.v</code>	$96 \rightarrow 1$	$v^\top(\cdot)$	把 96 维的「能量」压成 1 个分数

二、forward：算 context 和注意力权重

```
def forward(self, s, H, mask):
    energy = torch.tanh(self.W(s).unsqueeze(1) + self.U(H)) # [B, S, hid]
    scores = self.v(energy).squeeze(-1) # [B, S]
    scores = scores.masked_fill(~mask, float("-inf")) # PAD 位置打 -inf
```

```
alpha = F.softmax(scores, dim=-1) # [B, S] 和为 1
context = torch.bmm(alpha.unsqueeze(1), H).squeeze(1) # [B, hid]
return context, alpha
```

入参：

入参	形状	含义
s	[B, hid]	解码器上一步的隐藏态（代表「我现在想找什么」）
H	[B, S, hid]	编码器的全部输出（就是 Cell 4 的 outputs）
mask	[B, S]	布尔矩阵，True = 真实 token，False = PAD

第 1 步：算「能量」energy。

```
energy = torch.tanh(self.W(s).unsqueeze(1) + self.U(H)) # [B, S, hid]
```

拆成几小步看形状：

- `self.W(s)` : [B, hid] → [B, hid]。
- `.unsqueeze(1)` : 在第 1 维插一个长度为 1 的维度，[B, hid] → [B, 1, hid]。为什么插？为了和 H 的 S 个位置做广播相加。
- `self.U(H)` : [B, S, hid] → [B, S, hid]。
- 相加：[B, 1, hid] + [B, S, hid]，发生广播——那个长度 1 的维自动复制 S 份，跟每个源位置分别相加，结果 [B, S, hid]。等于「同一个 W_s ，跟每个 h_j 都配一次」。
- `torch.tanh(...)` : 逐元素过 tanh，形状不变 [B, S, hid]。

广播（broadcasting）：两个形状不完全相同的张量做运算时，PyTorch 自动把长度为 1 的维「拉伸」成对方长度，省得手动复制。这里 [B,1,hid] 被拉成 [B,S,hid]。

第 2 步：把能量压成每个位置一个分数。

```
scores = self.v(energy).squeeze(-1) # [B, S]
```

`self.v(energy)` : [B, S, hid] → [B, S, 1]；`.squeeze(-1)` : 去掉最后那个长度 1 的维，[B, S, 1] → [B, S]。现在 `scores` 是 [B, S]：每条样本、每个源位置一个原始分数。

第 3 步：屏蔽 PAD 位置。

```
scores = scores.masked_fill(~mask, float("-inf")) # PAD 位置打 -inf
```

`~mask` 对布尔矩阵取反（标出「哪些是 PAD」）；`masked_fill(~mask, -inf)` 把 PAD 位置分数填成负无穷。为什么填 `-inf`？因为下一步要 softmax， $\exp(-\infty) = 0$ ，所以 PAD 位置归一化后权重恰好为 0，等于「完全不看 PAD」。这正是 Cell 4 里「PAD 补 0 没关系，attention 会再屏蔽」那句注释的兑现点。

第 4 步：softmax 归一化成权重。

```
alpha = F.softmax(scores, dim=-1) # [B, S] 和为 1
```

softmax 把一组实数变成「和为 1 的正数」（概率分布）：

$$\alpha_j = \frac{\exp(\text{score}_j)}{\sum_k \exp(\text{score}_k)}$$

`dim=-1` 表示沿最后一维（S 方向）归一化，所以每条样本的 S 个权重加起来等于 1。形状不变 `[B, S]`。
`alpha` 就是「这一步对各源位置的关注度」。

第 5 步：加权求和得到 context。

```
context = torch.bmm(alpha.unsqueeze(1), H).squeeze(1) # [B, hid]
```

`alpha.unsqueeze(1)`：`[B, S] → [B, 1, S]`；`torch.bmm`（批量矩阵乘）对每条样本各做一次矩阵乘，`[B, 1, S] × [B, S, hid] → [B, 1, hid]`，本质就是 $\sum_j \alpha_j h_j$ （用权重把 S 个位置的隐藏态加权平均）；`.squeeze(1)`：`[B, 1, hid] → [B, hid]`。

`torch.bmm`（batch matrix multiply）专门处理「带 batch 维的成批矩阵乘」：第 0 维 B 是 batch，后两维才是真正相乘的矩阵。

出参：

出参	形状	含义
context	[B, hid]	这一步的上下文向量，喂给解码器
alpha	[B, S]	注意力权重（和为 1），可视化画的就是它，逆序任务里会呈反对角线

一句话总结：Cell 5 = 实现 Bahdanau 加性注意力。它吃解码器上一步隐藏态 `s` 和编码器全部输出 `H`，通过「W、U 变换 → 相加过 tanh → v 压成分数 → 屏蔽 PAD → softmax → 加权求和」这条流水线，产出为当前解码步定制的 context 和 alpha 权重。该 cell 只定义模块、没有自检打印——要等 Cell 6 把它装进解码器、Cell 7 串成整体后才真正跑起来。

Cell 6: 单步解码器（带 / 不带 attention 开关）

这是个「单步」解码器——一次只走一个时间步，由外层（Cell 7）反复调用走完整个序列。它有个开关 `use_attention`，一套代码同时支持「带 attention」和「不带 attention 的 baseline」两种模式，方便对比。

先立一个概念：

- `GRUCell`：注意和 Cell 4 的 `nn.GRU` 区分。`nn.GRU` 是「一口气吃整条序列」的版本；`nn.GRUCell` 是「只走一步」的版本——给它（当前输入 + 上一步隐藏态），它输出（这一步的新隐藏态）。解码器要在每一步插入 attention 计算，所以必须用能逐步控制的 `GRUCell`。

一、`__init__`：定义组件

```
class Decoder(nn.Module):
    def __init__(self, vocab, emb_dim, hid_dim, use_attention):
        super().__init__()
        self.use_attention = use_attention
        self.embed = nn.Embedding(vocab, emb_dim, padding_idx=PAD)
        self.attn = BahdanauAttention(hid_dim) if use_attention else None
        self.cell = nn.GRUCell(emb_dim + hid_dim, hid_dim)
        self.out = nn.Linear(hid_dim + hid_dim, vocab)
```

入参：

入参	含义
<code>vocab</code>	词表大小 13
<code>emb_dim</code>	嵌入维度 32
<code>hid_dim</code>	隐藏维度 96
<code>use_attention</code>	布尔开关： <code>True</code> =带注意力， <code>False</code> =baseline

创建的零件：

- `self.use_attention`：把开关存下来，`forward` 里要根据它走不同分支。
- `self.embed`：解码器自己的 Embedding（ 13×32 ），把「上一步输出的 token」查成向量。它和编码器的 `embed` 是两张独立的表（各学各的）。
- `self.attn`：注意力模块。又是三元表达式——`use_attention` 为真就造一个 Cell 5 的 `BahdanauAttention`，否则置 `None`。
- `self.cell = nn.GRUCell(emb_dim + hid_dim, hid_dim)`：单步 GRU。输入维是 `emb_dim + hid_dim = 32 + 96 = 128`——因为每步输入是「上一步 token 的 embedding(32)」和「context(96)」拼接。输出隐藏维 96。

- `self.out = nn.Linear(hid_dim + hid_dim, vocab)`：输出层，把 `[s ; context]`（`96 + 96 = 192` 维）投影到词表 13 维，得到每个 token 的分数（logits）。

为什么 `out` 的输入又把 `context` 拼一遍？这是 Bahdanau 论文的设计：预测下一个 token 时，既看新隐藏态 `s`，也直接再看一眼 `context`，让注意力信息更直接地影响输出。

二、step：走一个时间步

这个方法名叫 `step` 而不是 `forward`——因为它只是「一步」，真正的整句循环在 Cell 7 里。

```
def step(self, y_prev, s, H, mask, fixed_ctx):
    emb = self.embed(y_prev)                # [B, emb]
    if self.use_attention:
        context, alpha = self.attn(s, H, mask) # [B, hid], [B, S]
    else:
        context, alpha = fixed_ctx, None      # 始终同一个向量（瓶颈）
    s = self.cell(torch.cat([emb, context], dim=-1), s) # [B, hid] 新隐藏
    logits = self.out(torch.cat([s, context], dim=-1)) # [B, vocab]
    return logits, s, alpha
```

入参：

入参	形状	含义
<code>y_prev</code>	<code>[B]</code>	上一步的 token（teacher forcing 时是真值，推理时是上一步预测）
<code>s</code>	<code>[B, hid]</code>	上一步的隐藏态
<code>H</code>	<code>[B, S, hid]</code>	编码器全部输出（attention 要回看它）
<code>mask</code>	<code>[B, S]</code>	PAD 屏蔽掩码
<code>fixed_ctx</code>	<code>[B, hid]</code>	baseline 用的固定 context（编码器最后一步隐藏态）

第 1 步：上一步 token 查成向量。

```
emb = self.embed(y_prev)                # [B, emb]
```

`emb = self.embed(y_prev)`： `[B]` 的整数 token → `[B, 32]` 的向量。

第 2 步：拿到 `context`（两种模式的分岔点）。

```
if self.use_attention:
    context, alpha = self.attn(s, H, mask) # [B, hid], [B, S]
else:
    context, alpha = fixed_ctx, None      # 始终同一个向量（瓶颈）
```

这是整个 cell 最关键的对比：

- 带 attention：调用 Cell 5 的注意力模块，用当前隐藏态 `s` 回看整个 `H`，动态算出这一步专属的 `context`，同时拿到权重 `alpha` (`[B, S]`)。每一步的 `context` 都不同。
- baseline（不带）：`context` 恒等于 `fixed_ctx`——也就是编码器最后一步那个隐藏态，每一步都用同一个向量，`alpha` 设为 `None`。

这正是本章要演示的「信息瓶颈」：baseline 把整个源序列硬压进一个固定向量，序列越长越扛不住；attention 则每步动态回看，不受这个瓶颈限制。

第 3 步：更新隐藏态。

```
s = self.cell(torch.cat([emb, context], dim=-1), s) # [B, hid] 新隐藏
```

`torch.cat([emb, context], dim=-1)`：沿最后一维拼接，`[B, 32]` 和 `[B, 96]` 拼成 `[B, 128]`；
`self.cell(拼接输入, s)`：GRUCell 吃「当前输入(128)」和「上一步隐藏(96)」，输出新隐藏态 `[B, 96]`。
新的 `s` 覆盖旧的，下一步还会接着用。

`torch.cat` vs 之前的 `unsqueeze`：`cat` 是「拼接」，把两个张量在某一维首尾接起来（总长度相加）；`unsqueeze` 是「插一个长度 1 的新维」。这里 $32+96=128$ 是拼接。

第 4 步：算 logits。

```
logits = self.out(torch.cat([s, context], dim=-1)) # [B, vocab]
```

`torch.cat([s, context], dim=-1)`：把新隐藏态 `s` (96) 和 `context`(96) 拼成 `[B, 192]`；过 `self.out` 投到词表维得 `[B, 13]`——每条样本对 13 个 token 各一个分数，最大的那个就是模型这一步「想输出」的 token。

logits：神经网络分类输出的原始分数（未归一化）。后面算 loss 时交给 `cross_entropy`（内部会做 softmax），或推理时直接取最大值（argmax）。

出参：

出参	形状	含义
logits	[B, 13]	这一步对各 token 的打分
s	[B, 96]	这一步的新隐藏态，喂给下一步
alpha	[B, S] 或 None	注意力权重（baseline 为 None），可视化用

一句话总结：Cell 6 = 定义单步解码器 `Decoder.step`。每步做四件事：① 上一步 token 查 embedding → ② 拿 context（带 attention 就动态算、baseline 就用固定向量，这是两种模式的唯一分岔）→ ③ `[emb;context]` 喂 GRUCell 更新隐藏态 → ④ `[s;context]` 过输出层得 logits。

Cell 7: 组装 Seq2Seq + 训练 / 评估函数

这是把前面零件串成整体的一节：`Seq2Seq` 类（编码器+解码器拼起来，跑完整句），外加 `train`（训练循环）和 `seq_accuracy`（评估）两个函数。

一、Seq2Seq 类

```
class Seq2Seq(nn.Module):
    def __init__(self, vocab, emb_dim, hid_dim, use_attention):
        super().__init__()
        self.encoder = Encoder(vocab, emb_dim, hid_dim)
        self.decoder = Decoder(vocab, emb_dim, hid_dim, use_attention)

    def forward(self, src, src_len, tgt_in, return_attn=False):
        H, h_n = self.encoder(src, src_len)      # H: [B, S, hid], h_n: [1, B, hid]
        s = h_n.squeeze(0)                       # [B, hid]
        fixed_ctx = s
        mask = torch.arange(src.size(1)).unsqueeze(0) < src_len.unsqueeze(1)  # [B, S]
        logits_seq, attn_seq = [], []
        for t in range(tgt_in.size(1)):         # T 步
            logits, s, alpha = self.decoder.step(tgt_in[:, t], s, H, mask, fixed_ctx)
            logits_seq.append(logits)
            if return_attn and alpha is not None:
                attn_seq.append(alpha)
        logits = torch.stack(logits_seq, dim=1)  # [B, T, vocab]
        attn = torch.stack(attn_seq, dim=1) if (return_attn and attn_seq) else None
        return logits, attn
```

`__init__` 就是创建 Cell 4 的 `Encoder` 和 Cell 6 的 `Decoder` 各一个，`use_attention` 直接透传给解码器——所以整个模型带不带注意力，由这个开关决定。

`forward` 入参：

入参	形状	含义
<code>src</code>	<code>[B, S]</code>	编码器输入（数字 token，右侧补 PAD）
<code>src_len</code>	<code>[B]</code>	每条序列的真实长度（不含 PAD）
<code>tgt_in</code>	<code>[B, T]</code>	解码器输入（SOS + 逆序）， <code>T = Smax+1</code>
<code>return_attn</code>	bool	是否额外返回注意力矩阵，可视化时才开

① 编码。

```
H, h_n = self.encoder(src, src_len)      # H: [B, S, hid], h_n: [1, B, hid]
s = h_n.squeeze(0)                      # [B, hid]
fixed_ctx = s
```

`H, h_n = self.encoder(...)` 拿到全位置输出 `H` 和最后一步隐藏 `h_n`；`s = h_n.squeeze(0)` 去掉长度 1 的层数维，`[1, B, hid] → [B, hid]`，作为解码器初始隐藏态（用编码器读完整句的记忆给解码器起步）；`fixed_ctx = s` 是 baseline 用的固定 context（正是编码器最后一步）。

② 造 mask。

```
mask = torch.arange(src.size(1)).unsqueeze(0) < src_len.unsqueeze(1)  # [B, S]
```

这行用广播一次性生成 PAD 掩码：

- `torch.arange(src.size(1))`：`[0, 1, ..., S-1]`，形状 `[S]`；`.unsqueeze(0) → [1, S]`（一行位置下标）。
- `src_len.unsqueeze(1) → [B, 1]`（一列真实长度）。
- `<` 比较时广播成 `[B, S]`：第 `i` 行第 `j` 列 = `(j < src_len[i])`，即「位置 `j` 在第 `i` 条的真实长度内吗」——是就 `True`（真实 token），否则 `False`（PAD）。

举例 `src_len=[6,4,5]`、`S=6`，第 1 行（长度 4）得 `[T,T,T,T,F,F]`，正好标出后两个是 PAD。这个 mask 一路传给解码器里的 attention。

③ 逐步解码循环。

```
logits_seq, attn_seq = [], []
for t in range(tgt_in.size(1)):      # T 步
    logits, s, alpha = self.decoder.step(tgt_in[:, t], s, H, mask, fixed_ctx)
    logits_seq.append(logits)
    if return_attn and alpha is not None:
        attn_seq.append(alpha)
```

循环 `T` 次（`tgt_in.size(1)=T`）。每步把 `tgt_in[:, t]`（所有样本第 `t` 列的 token，`[B]`）喂给 `step`——这就是 teacher forcing：不管模型上一步预测对不对，第 `t` 步一律喂真实的 `tgt_in[:, t]`，让训练更稳更快。`s` 每步被更新并带入下一步；每步 `logits`（`[B, 13]`）收进列表，要可视化时 `alpha`（`[B, S]`）也收进列表。

Python：`[]` 是空列表，`.append(x)` 往后追加一个元素。这里把 `T` 步结果先攒在列表里，循环结束再堆叠成张量。

④ 堆叠成结果。

```
logits = torch.stack(logits_seq, dim=1)    # [B, T, vocab]
attn = torch.stack(attn_seq, dim=1) if (return_attn and attn_seq) else None
return logits, attn
```

`torch.stack(列表, dim=1)`：把 T 个 $[B, 13]$ 沿新的第 1 维堆成 $[B, T, 13]$ ；`attn` 同理堆成 $[B, T, S]$ ，不需要时为 `None`。

`torch.stack` vs `torch.cat`：`stack` 新增一个维度再堆（ T 个 $[B, 13] \rightarrow [B, T, 13]$ ）；`cat` 是在已有维度上接长。这里要新造「时间步」这一维，所以用 `stack`。

出参：

出参	形状	含义
<code>logits</code>	$[B, T, \text{vocab}]$	每步对各 token 的打分
<code>attn</code>	$[B, T, S]$ 或 <code>None</code>	注意力对齐矩阵（不需要时为 <code>None</code> ）

二、`train` 函数：训练循环

```
def train(model, steps=2000, batch_size=64, lr=2e-3, log_every=400):
    opt = torch.optim.Adam(model.parameters(), lr=lr)
    model.train()
    for step in range(1, steps + 1):
        src, src_len, tgt_in, tgt_out = make_batch(batch_size)
        logits, _ = model(src, src_len, tgt_in)
        loss = F.cross_entropy(logits.reshape(-1, VOCAB), tgt_out.reshape(-1),
                               ignore_index=PAD)

        opt.zero_grad()
        loss.backward()
        opt.step()
        if step % log_every == 0 or step == 1:
            print(f" step {step:4d}/{steps}  loss = {loss.item():.4f}")
    return model
```

入参：

入参	含义
<code>model</code>	要训的模型
<code>steps</code>	训多少步（默认 2000，Cell 8 传 1500）
<code>batch_size</code>	每批多少条样本（默认 64）
<code>lr</code>	学习率（默认 $2e-3$ ）
<code>log_every</code>	每隔多少步打印一次（默认 400）

出参:

出参	含义
model	训练好的模型（原地训练，返回的就是传入的那个对象）

这是标准的训练循环，拆开看是这几步：

- `opt = torch.optim.Adam(...)`：创建优化器 Adam，负责按梯度更新参数。`model.parameters()` 把模型所有可学习参数交给它，`lr` 是学习率（步子大小）。
- `model.train()`：切到「训练模式」（影响 dropout、BN 等；本模型没有，但写上和规范）。
- 循环每步：
 - i. `make_batch(batch_size)`：现造一批新数据（注意没给 `seed`，每步数据都不同——相当于无限数据）。
 - ii. `logits, _ = model(...)`：前向，拿 `[B, T, 13]` 的预测；`_` 丢弃这里用不到的 `attn`。
 - iii. 算 loss: `logits.reshape(-1, VOCAB)` 把 `[B, T, 13]` 摊平成 `[B*T, 13]`（`-1` 表示「这一维大小自动算」），当成 `B*T` 个独立分类样本；`tgt_out.reshape(-1)` 把真值摊平成 `[B*T]`；`F.cross_entropy(...)` 算交叉熵损失。`ignore_index=PAD` 是关键——目标里是 `PAD(0)` 的位置不计入 loss（PAD 是凑数的，预测它什么都无所谓，不该被惩罚）。
 - iv. 反向传播三件套：`opt.zero_grad()` 清空上一步残留梯度（PyTorch 梯度默认累加，不清会叠加错）；`loss.backward()` 自动求导算梯度；`opt.step()` 按梯度更新参数。
 - v. 按 `log_every` 打印进度。`loss.item()` 把单元素张量取成普通 Python 数。

`{step:4d}` / `{loss:.4f}` 是格式化：`4d` 占 4 位整数右对齐，`.4f` 保留 4 位小数。

三、seq_accuracy 函数：评估

```
@torch.no_grad()
def seq_accuracy(model, n_batches=20, batch_size=128, min_len=4, max_len=12):
    model.eval()
    correct = total = 0
    for _ in range(n_batches):
        src, src_len, tgt_in, tgt_out = make_batch(batch_size, min_len, max_len)
        logits, _ = model(src, src_len, tgt_in)
        pred = logits.argmax(-1)                                # [B, T]
        valid = tgt_out != PAD                                  # [B, T] 布尔
        ok = ((pred == tgt_out) | ~valid).all(dim=1)            # [B] 每条是否全对
        correct += ok.sum().item()
        total += src.size(0)
    return correct / total
```

入参:

入参	含义
<code>model</code>	要评估的模型
<code>n_batches</code>	评估几个 batch 求平均（默认 20）
<code>batch_size</code>	每批多少条样本（默认 128）
<code>min_len</code> / <code>max_len</code>	测试序列长度区间（默认 4-12, Cell 10 按桶传入不同区间）

出参:

出参	形状	含义
返回值	标量 (float)	0-1 之间的 exact-match 准确率（整句全对才算对）

`@torch.no_grad()`：装饰器，告诉 PyTorch 这个函数里不需要算梯度（评估只看结果不训练），省显存、跑得快。`model.eval()`：切到「评估模式」。

逻辑——整句全对才算对（exact match，比「逐字符准确率」严格）：

- `pred = logits.argmax(-1)`：沿最后一维（13 个 token）取最大值下标，`[B, T, 13] → [B, T]`，即每步贪心预测的 token。
- `valid = tgt_out != PAD`：`[B, T]` 布尔，标出哪些位置是真实的（非 PAD）。
- `ok = ((pred == tgt_out) | ~valid).all(dim=1)`：逐位判断「预测对了 或 这本就是 PAD 位（不计较）」，再 `.all(dim=1)` 沿时间维要求整句每个位置都满足，得 `[B]` 布尔。用 `| ~valid` 的巧思：PAD 位置直接放行（算「对」），只真正考核有效位置。
- `correct += ok.sum().item()` 累加本批对的条数；`total += src.size(0)` 累加样本总数；最后返回 `correct / total`。

`& / | / ~` 是张量的逐元素与/或/非；`.all(dim=1)` 沿第 1 维（时间）做「全部为真」的归约，把 `[B,T]` 压成 `[B]`。

一句话总结：Cell 7 = 把零件组装成可训练的完整模型 + 配套训练/评估函数。`Seq2Seq.forward` 干三件事：编码 → 造 PAD mask → teacher-forcing 逐步解码并堆成 `[B,T,vocab]`；`train` 是「造数据 → 前向 → 交叉熵（忽略 PAD）→ 反向 → 更新」的标准循环；`seq_accuracy` 用「整句全对才算对」的严格口径评估。最该记住两点：teacher forcing（每步喂真值 `tgt_in[:,t]`）和 `ignore_index=PAD` / `| ~valid`（PAD 既不算 loss 也不计入准确率）。

Cell 8: 控制变量，训练两个模型

这个 cell 很短，就是「按下开始键」：分别训练两个模型——一个带 attention、一个不带（baseline），后面好对比。

```
import time

torch.manual_seed(0)
print("训练【带 attention】的 seq2seq ...")
t0 = time.time()
model_attn = Seq2Seq(VOCAB, emb_dim=32, hid_dim=96, use_attention=True)
train(model_attn, steps=1500)
print(f" 用时 {time.time() - t0:.1f}s\n")

torch.manual_seed(0)
print("训练【不带 attention】的 baseline (context 固定为编码器最后一步的隐藏状态) ...")
t0 = time.time()
model_plain = Seq2Seq(VOCAB, emb_dim=32, hid_dim=96, use_attention=False)
train(model_plain, steps=1500)
print(f" 用时 {time.time() - t0:.1f}s")
```

- `import time`：只为给训练计时。
- `torch.manual_seed(0)`（两次都有，关键）：设定全局随机种子为 0，它影响两处随机——模型参数初始化、以及 `train` 里 `make_batch` 造数据的随机性。为什么训练前各设一次、且都设成 0？为了公平对比——让两个模型从「同样的随机起点、喂同样序列的数据」出发，唯一差别只剩 `use_attention` 这一个开关。否则万一是初始化运气导致的差异，就说不清是不是 attention 的功劳了。这是对照实验的标准做法：控制变量。为什么不能只在开头设一次？因为 `manual_seed` 设的是全局随机状态，它会被后续每次随机操作一点点「消耗」掉：建完 `model_attn` 又训练 1500 步后，全局状态已经被推得很远，轮到 `model_plain` 时拿到的是「一堆用剩的随机数」，初始化和数据都在不可控地错开。在每个模型前各重置回 0，等于把随机状态拨回同一个干净起点，消除这种漂移。一个诚实的小补充：带 attention 的模型多了 $W/U/v$ 几组参数，初始化时比 baseline 多抽了些随机数，所以严格说两者训练数据流不会逐字节完全相同（分岔点之后随机序列就错开了）；但两者都从 `seed=0` 这个相同起点出发，已经消除了最大那个系统性偏差，这就是实践中保证公平 + 可复现的通用手法。
- `t0 = time.time()`：记录当前时刻（从 1970 年起的秒数），训练结束后用 `time.time() - t0` 算耗时。
- 建模型 + 训练：`model_attn`（`use_attention=True`，带注意力）和 `model_plain`（`use_attention=False`，baseline，context 恒为编码器最后一步隐藏态）。两者超参完全一样（`emb_dim=32`，`hid_dim=96`），各 `train(..., steps=1500)`，其余参数用默认值，训练中按 `log_every` 打印 loss。
- 打印耗时：`:.1f` 保留 1 位小数，`\n` 在带 attention 那次末尾多打一个空行隔开两段。

`model_attn` / `model_plain` 是 cell 顶层的全局变量，训练后一直留在内存，供 Cell 9（可视化）、Cell 10（长度分桶对比）直接取用。

这个 cell 没定义函数，就是一段顺序执行的脚本。它的「产物」是两个训练好的全局模型对象，同时打印两组训练日志（每隔若干步的 loss 变化 + 总耗时）。理论上带 attention 那个 loss 会降得更低、更快收敛——但真正的对比要看 Cell 9 和 Cell 10。

一句话总结：Cell 8 = 用相同种子、相同超参，分别训练带 attention 和 baseline 两个模型，只让 `use_attention` 这一个变量不同，为后两个 cell 备好 `model_attn` / `model_plain`。核心是 `torch.manual_seed(0)` 保证的「控制变量」。

Cell 9: 可视化注意力对齐（反对角线）

这个 cell 把训练好的 `model_attn` 在一条样本上跑一遍，把注意力权重画成热力图——逆序任务里它应该呈现一条漂亮的反对角线，肉眼验证「attention 真的学会了回看正确的输入位置」。

```
import matplotlib.pyplot as plt

src, src_len, tgt_in, tgt_out = make_batch(1, min_len=8, max_len=8, seed=7)
model_attn.eval()
with torch.no_grad():
    logits, attn = model_attn(src, src_len, tgt_in, return_attn=True)
pred = logits.argmax(-1)[0] # [T]

src_digits = (src[0] - DIGIT0).tolist() # 还原成 0..9 便于看
L = src_len.item()
A = attn[0, :L, :L].numpy() # [L, L] 只取有效部分

print("源序列(数字):", src_digits[:L])
print("目标(逆序) :", (tgt_out[0, :L]).sub(DIGIT0).tolist())
print("模型预测      :", (pred[:L]).sub(DIGIT0).tolist())

plt.figure(figsize=(5, 4))
plt.imshow(A, cmap="viridis", aspect="auto")
plt.xlabel("source position j") # matplotlib 标签一律英文，避免乱码
plt.ylabel("decoder step t")
plt.title("Bahdanau Attention Alignment (reverse task)")
plt.colorbar(label="attention weight")
plt.tight_layout()
plt.show()
print("→ 亮点大致沿反对角线分布：解码第 t 步把注意力压在源序列第 (L-1-t) 个位置附近")
print("（有的训练结果会整体偏移一格——提前一步取数、存进隐藏状态下一步再用，效果等价），")
print("正是逆序任务需要的对齐——attention 自己学会了『回看正确的输入位置』。")
```

一、准备一条样本并前向。

- `make_batch(1, min_len=8, max_len=8, seed=7)`：造1条样本（B=1），长度固定为8（min=max=8，画出来是规整方阵好看），`seed=7` 固定结果可复现。
- `model_attn.eval()`：切评估模式。
- `with torch.no_grad():`：这个代码块内不算梯度（只看结果不训练）。`with` 是 Python 的上下文管理器，块内临时关闭梯度，出块自动恢复。

- `model_attn(..., return_attn=True)`：关键——打开 `return_attn` 开关，让 Cell 7 的 forward 额外返回注意力矩阵 `attn`（形状 `[B, T, S] = [1, 9, 8]`，`T=Smax+1=9`）。
- `pred = logits.argmax(-1)[0]`：`logits` 是 `[1, T, 13]`，`argmax(-1)` 沿词表维取最大得 `[1, T]`，`[0]` 取出唯一那条 → `[T]`，即每步预测 token。

二、还原成人类可读的数字。

- `src[0] - DIGIT0`：`src` 存的是 token（数字+3），减回 `DIGIT0(=3)` 还原成真数字 0–9；`.tolist()` 转成列表。
- `L = src_len.item()`：取出真实长度（这里 8）。`.item()` 把单元素张量取成普通数。
- `A = attn[0, :L, :L].numpy()`：从 `[1, T, S]` 里取第 0 条、只截前 L 行 L 列的有效部分，转 numpy 给 matplotlib 用。为什么截 `[:L, :L]`？`T=9` 比 `L=8` 多一步（末尾要预测 EOS），`S` 方向也可能有 PAD；只看 `L×L` 有效区域画出来才干净。`A[t, j]` = 解码第 `t` 步对源位置 `j` 的注意力权重。

三、打印三行做对照。把「源序列 / 目标(逆序) / 模型预测」并排打出来肉眼核对。`.sub(DIGIT0)` 是 `-DIGIT0` 的方法写法（减 3 还原数字）。模型学好了的话，目标和预测应一致，且都是源序列的逆序。

四、画热力图。

- `plt.figure(figsize=(5, 4))`：新建画布，宽 5 高 4 英寸。
- `plt.imshow(A, cmap="viridis", aspect="auto")`：核心——把矩阵 `A` 当图像画，每个格子 `A[t, j]` 的数值映射成颜色（`viridis` 配色：深紫=低、黄=高），`aspect="auto"` 让格子自动拉伸填满。
- `xlabel / ylabel / title`：坐标轴和标题。x 轴是源位置 `j`，y 轴是解码步 `t`。
- `plt.colorbar(...)`：画右侧色条，标明颜色对应的权重大小。`plt.tight_layout()` 调边距、`plt.show()` 渲染显示。

怎么读这张图：y 轴从上到下是解码第 0、1、2… 步，x 轴从左到右是源位置 0、1、2…。某格越黄，表示该解码步越关注那个源位置。逆序任务里，解码第 `t` 步要输出源序列第 $(L-1-t)$ 个数，所以亮点会沿反对角线（右上到左下）分布。偶尔整体偏移一格（提前一步取数、存进隐藏态下一步再用）也是等价的，不用担心。

一句话总结：Cell 9 = 拿训练好的 `model_attn` 在一条定长(8)样本上前向，用 `return_attn=True` 取出注意力矩阵，截出有效的 `L×L` 部分用 `imshow` 画成热力图，并打印「源/目标/预测」三行做对照。看点是那条反对角线——它直观证明 attention 自己学会了「解码第 `t` 步去回看源序列第 $(L-1-t)$ 个位置」。

Cell 10：定量对比，展示瓶颈效应

本章最后一个 cell，从定量角度收尾：把测试数据按序列长度分桶，对比两个模型的准确率，直观展示「序列越长，baseline 越扛不住」这个信息瓶颈——正是 attention 当年被发明出来要解决的问题。

```

buckets = [(4, 5), (6, 7), (8, 9), (10, 12)]
acc_attn, acc_plain = [], []
for lo, hi in buckets:
    acc_attn.append(seq_accuracy(model_attn, min_len=lo, max_len=hi))
    acc_plain.append(seq_accuracy(model_plain, min_len=lo, max_len=hi))

labels = [f"{lo}-{hi}" for lo, hi in buckets]
for lab, a, p in zip(labels, acc_attn, acc_plain):
    print(f" 长度 {lab:5s}  attention={a:.2%}  baseline={p:.2%}")

x = range(len(buckets))
plt.figure(figsize=(6, 4))
plt.plot(list(x), acc_attn, "o-", label="with attention")
plt.plot(list(x), acc_plain, "s--", label="no attention (baseline)")
plt.xticks(list(x), labels)
plt.xlabel("source length bucket")
plt.ylabel("exact-match accuracy")
plt.title("Why Attention: accuracy vs sequence length")
plt.ylim(0, 1.05)
plt.legend()
plt.grid(alpha=0.3)
plt.tight_layout()
plt.show()

```

一、按长度分桶跑准确率。

- `buckets` : 列表, 每个元素是二元组 `(lo, hi)`, 表示一个长度区间。共 4 个桶: 短(4-5)、中(6-7)、较长(8-9)、长(10-12)。
- `acc_attn, acc_plain = [], []` : 两个空列表, 分别存两个模型在各桶的准确率。
- 循环每个桶 `(lo, hi)` : `seq_accuracy(model_attn, min_len=lo, max_len=hi)` 调 Cell 7 的评估函数, 只在这个长度区间生成测试数据评估, 返回 exact-match 准确率; 对 baseline 同样跑一次; 各 `append` 进对应列表。

Python: `for lo, hi in buckets` 是「解包遍历」——每轮自动把元组 `(4,5)` 拆成 `lo=4, hi=5`。循环结束后两个列表各长 4, 如 `[0.99, 0.97, 0.90, 0.75]`。

关键设计: 用长度区间控制难度。同一个训练好的模型, 喂不同长度的测试串, 看它在长串上是否退化。

二、打印对比表。

- `labels` : 列表推导式, 把每个桶变成字符串标签 `["4-5", "6-7", "8-9", "10-12"]`。列表推导式 [表达式 `for x in 序列`] 是 Python 简洁地「边遍历边造新列表」的写法。
- `zip(labels, acc_attn, acc_plain)` : 把三个等长列表「拉链式」配对, 每轮同时取出一组 (标签, attn准确率, baseline准确率)。
- 打印格式: `{lab:5s}` 标签占 5 字符宽, `{a:.2%}` 把小数转成百分比并保留 2 位 (如 `0.973` → `97.30%`)。

三、画折线图。

- `x = range(len(buckets))` : `range(4)` → `0,1,2,3` , 作为横坐标的位置。
- `plt.plot(list(x), acc_attn, "o-", label="with attention")` : 画带 attention 的曲线。"o-" 是样式串——o 圆点标记 + - 实线。
- `plt.plot(..., acc_plain, "s--", ...)` : 画 baseline, "s--" = 方块标记 + 虚线, 和上面区分开。
- `plt.xticks(list(x), labels)` : 把横轴刻度 `0,1,2,3` 换成可读的 `4-5, 6-7, ...` 标签。
- `plt.ylim(0, 1.05)` : y 轴范围锁定 0-1.05 (准确率是 0-1, 留一点顶部空白)。
- `plt.legend()` 显示图例 (区分两条线) ; `plt.grid(alpha=0.3)` 画淡淡的网格线 (`alpha=0.3` 是透明度, 半透明不抢眼) ; `tight_layout()` / `show()` 调边距、显示。

怎么读这张图：横轴从左到右序列越来越长，纵轴是整句准确率。两条线在短桶都接近 1；随着变长，baseline (虚线) 明显下滑，而 attention (实线) 基本保持平稳。两条线的差距越往右越大，就是「瓶颈」的可视化。

一句话总结：Cell 10 = 把测试集按长度分成 4 个桶，分别评估两个模型，用表格 + 折线图对比准确率。核心看点：序列越长，固定 context 的 baseline 准确率掉得越快，而 attention 基本稳住——这就是用数据印证「attention 解决了 RNN seq2seq 的信息瓶颈」。

补充：真正的自回归推理

Cell 9 和 Cell 10 其实都还在用 teacher forcing。它俩评估时都调了 `model(src, src_len, tgt_in)`，而这个 `tgt_in` 是 `make_batch` 给的真值 (SOS + 正确逆序)。也就是说，解码器每一步拿到的「上一个 token」都是标准答案——哪怕它前一步其实猜错了，下一步也会被「纠正」回正轨。这在训练时是有意为之 (让训练更稳、更快)，但拿来评估有点「开卷考试」的味道。

真实部署时根本没有 `tgt_in`——它正是我们要生成的东西 (你要是已经有正确逆序，还要模型干嘛)。所以真正的推理只能这样做：第一步喂 SOS，让模型输出一个 token；把这个预测接回去当第二步的输入，再输出下一个；如此一步接一步，直到模型自己输出 EOS (或到达长度上限)。这个「拿自己上一步的输出当下一步的输入」的过程就叫自回归 (autoregressive) 解码，也叫 free-running。

下面这段就是一个最小的贪心自回归解码器：给一串数字，让训练好的模型自己把它倒过来。它没有合并进 05.ipynb——在 ipynb 末尾手动新增一个 cell，把下面这段复制进去执行即可 (它依赖 Cell 7 定义的 Seq2Seq / Cell 3 的 `make_batch` 常量、以及 Cell 8 训练好的 `model_attn` / `model_plain`，所以要在前面 10 个 cell 全部跑完之后再跑)：

```
# =====
# 补充：真正的自回归推理——不再 teacher forcing (手动新增一个 cell 粘进去跑)
# =====
# Cell 9/10 评估时调用 model(src, src_len, tgt_in)，里头喂的是【真值】tgt_in—
# 这叫 teacher forcing：每一步解码器都拿到正确的「上一个 token」，哪怕它前一步猜错了。
# 真实部署时我们【没有】答案 tgt_in (它正是要生成的东西)，只能把模型【自己】上一步的
```

```
# 预测接回来当下一步输入，一个 token 一个 token 地往外输出——这就是自回归(autoregressive)推理。

@torch.no_grad()
def greedy_decode(model, digits, max_len=20):
    """给一串数字 (Python list, 元素 0..9)，返回模型自回归生成的输出数字串。
    与 Cell 9/10 的区别：这里【不喂任何真值】，每步把上一步的预测接回来当输入。"""
    model.eval()
    L = len(digits)
    # 数字 -> token (+DIGIT0)，再加一个 batch 维 -> [1, L]
    src = torch.tensor([[d + DIGIT0 for d in digits]], dtype=torch.long)
    src_len = torch.tensor([L])
    # 编码一次 (和 forward 里那步一模一样)
    H, h_n = model.encoder(src, src_len)
    s = h_n.squeeze(0)
    fixed_ctx = s
    mask = torch.arange(src.size(1)).unsqueeze(0) < src_len.unsqueeze(1) # [1, L] 全 True
    y = torch.tensor([SOS])
    out_digits = []
    for _ in range(max_len):
        logits, s, _ = model.decoder.step(y, s, H, mask, fixed_ctx)
        y = logits.argmax(-1)
        tok = y.item()
        if tok == EOS:
            break
        out_digits.append(tok - DIGIT0)
    return out_digits

# 随便给一串数字，看两个模型自回归地把它倒过来
demo = [3, 1, 4, 1, 5, 9, 2, 6]
print("输入数字串          :", demo)
print("正确逆序          :", demo[::-1])
print("attention 模型自回归输出:", greedy_decode(model_attn, demo))
print("baseline 模型自回归输出:", greedy_decode(model_plain, demo))
```

逐块说明：

入参 / 出参。

名称	形状 / 类型	含义
入参 <code>model</code>	<code>Seq2Seq</code>	训练好的模型（带或不带 attention 都行）
入参 <code>digits</code>	Python list	要反转的数字串，元素是 0-9
入参 <code>max_len</code>	int	最多生成多少步，防止不出 EOS 时死循环
出参	Python list	模型自回归输出的数字串（已从 token 还原成 0-9）

编码部分和 `forward` 完全一样：数字串转成 `src`（`[1, L]`）→ 编码器跑一遍拿到 `H`、`h_n` → `s = h_n.squeeze(0)` 当解码器初始隐藏态 → 造一个全 True 的 `mask`（这里没有 PAD）。

解码循环是关键差异所在。对比 Cell 7 的 `forward`：

	Cell 7 forward (训练/评估)	greedy_decode (真实推理, 手动新增 cell)
第 t 步喂给 step 的 token	tgt_in[:, t] ——真值	y ——模型自己上一步的预测
循环几步	固定 T 步 (tgt_in 有多长走多长)	直到输出 EOS 或到 max_len
错了会怎样	不影响下一步 (下一步仍喂真值)	误差会累积——错一步, 后面拿着错的输入继续错

循环里那行 `y = logits.argmax(-1)` 取当前步概率最大的 token, 然后这个 **y** 直接成为下一次 **step** 的 **y_prev** ——这就是「自回归」三个字的全部含义。`if tok == EOS: break` 把「什么时候停」的决定权交给模型自己。

为什么 **baseline** 在这里更容易暴露问题。teacher forcing 评估 (Cell 10) 对 **baseline** 已经不太友好了; 换成自回归更残酷——**baseline** 把整句压进一个固定向量, 长串本来就记不全, 一旦某步输出错误, 错误 token 又被接回去喂给下一步, 误差不断累积。这种「训练时一直喂真值、推理时只能喂自己的预测」带来的落差, 有个专门的名字叫 **exposure bias** (曝光偏差)。带 **attention** 的模型每步都能回看原始输入, 受这个问题的影响小得多, 所以在自回归下通常仍能把 **demo** 干净地倒过来。

顺便提一句: 这里用的是最简单的贪心解码 (每步只取概率最大的那个)。实际 LLM 推理还有 **temperature**、**top-k**、**top-p**、**beam search** 等方法, 但「拿上一步输出当下一步输入」这个自回归骨架是完全一样的——这也是第 2 章生成参数那套采样策略真正发挥作用的地方。

一句话总结: 这一节给的是一个最小的贪心自回归解码器, 揭示 Cell 9/10 其实都在 teacher forcing (喂真值), 而真实推理必须把模型自己的预测逐步接回来、直到它输出 **EOS**。它既给了你「拿训练好的模型推理一串自定义输入」的现成代码 (手动新增 cell 粘进去跑), 也直观展示了 **exposure bias** 下 **baseline** 比 **attention** 更容易崩。

至此第五章实战的 Cell 2-10 全部讲完。回顾整条主线: 手算 RNN (Cell 2) 建立直觉 → 造数据 (Cell 3) → 编码器 (Cell 4) → 注意力 (Cell 5) → 解码器 (Cell 6) → 组装训练 (Cell 7) → 控制变量训两个模型 (Cell 8) → 可视化对齐 (Cell 9) → 定量对比瓶颈 (Cell 10); 最后补充一节用自回归推理点出 teacher forcing 与真实生成的差别。配合正文第 5 章: 从 RNN 到 attention 的原理推导一起看, 「为什么」和「怎么做」就都齐了。